

You are right to correct the direction. **The sections are for controlling development, not for designing the Headquarters as disconnected pieces.**

If we fully design Central Ops, then later design SOC, then later realise the campaign route between them is poor, we will keep rebuilding. The correct method is:

Design Phantom Q Headquarters as one complete operational system first. Then use the sections only to control how we construct and test that already-defined system.

That changes the approach significantly.

1. We need one Headquarters master design

Before serious Unity construction, we need one source of truth containing all of these simultaneously:

```
PHANTOM Q HQ MASTER DESIGN
|
├─ PURPOSE OF THE BUILDING
├─ PHYSICAL LAYOUT
├─ DEPARTMENT RELATIONSHIPS
├─ CAMPAIGN MOVEMENT
├─ CHARACTER MOVEMENT
├─ CLEARANCE PROGRESSION
├─ ROOM FUNCTIONS
├─ INTERACTIONS
├─ VISUAL LANGUAGE
├─ PLAYER HANDLING / CAMERA
└─ UNITY TECHNICAL STRUCTURE
```

These cannot be designed independently because they affect one another.

For example:

```
SOC location
  ↓
affects Incident 04
  ↓
affects Incident 06 investigation
  ↓
affects Red Team adjacency
  ↓
affects player walking distance
  ↓
affects camera
  ↓
affects corridor dimensions
```

↓

That is the loop we need to think through **before building**.

2. The building must tell the Phantom Q story

The Headquarters should have meaning even if every tutorial label disappeared.

The architecture itself should communicate:

```
ENTRY
↓
IDENTITY
↓
NORMAL OPERATIONS
↓
LEARNING / SPECIALISATION
↓
SECURITY OPERATIONS
↓
ADVANCED ENGINEERING
↓
QUANTUM
↓
HIGH-TRUST / SECURE CORE
```

So the building represents the player's progression.

At the beginning, the player enters as an outsider.

By Incident 06, they are moving through areas they could only see from behind access-controlled doors earlier.

That means **clearance progression is physically embedded into the architecture**.

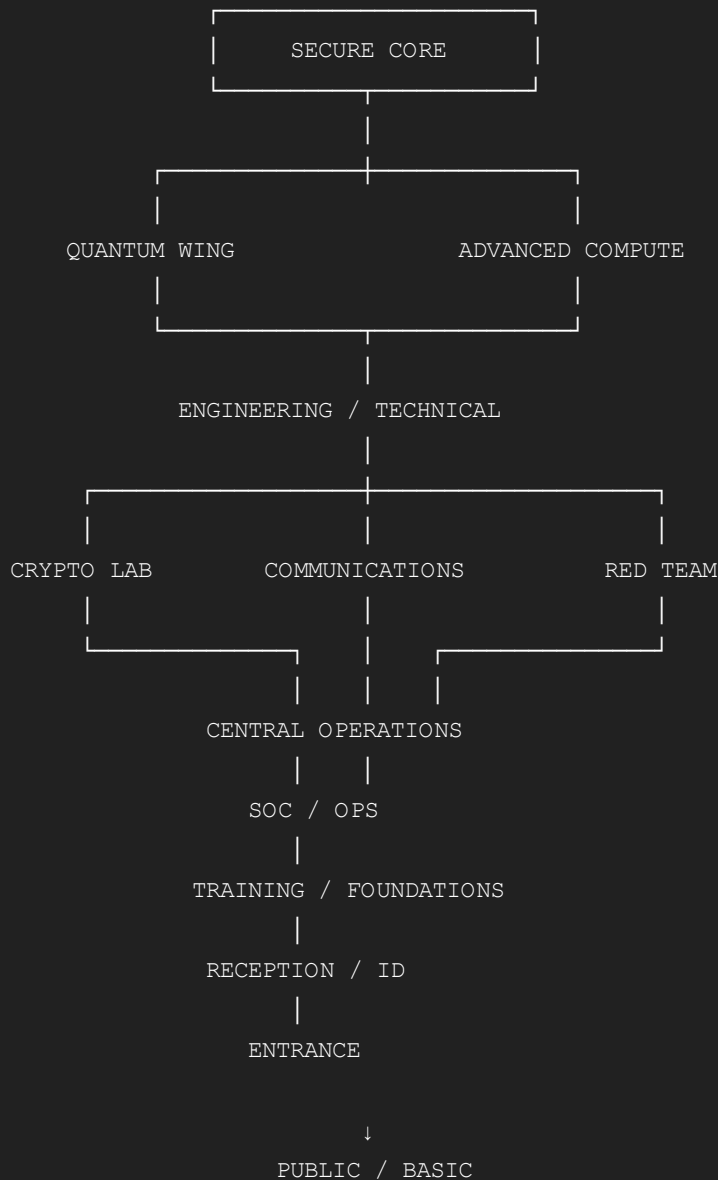
3. There should be a security-depth gradient

Rather than scattering departments randomly because there is empty floor space, I would organise HQ around increasing operational sensitivity.

Conceptually:

DEEP / RESTRICTED

↑



I am **not** locking that exact drawing as the floor plan yet.

What I am locking conceptually is the gradient:

The further into Phantom Q's capabilities the learner progresses, the deeper their authorised access into HQ becomes.

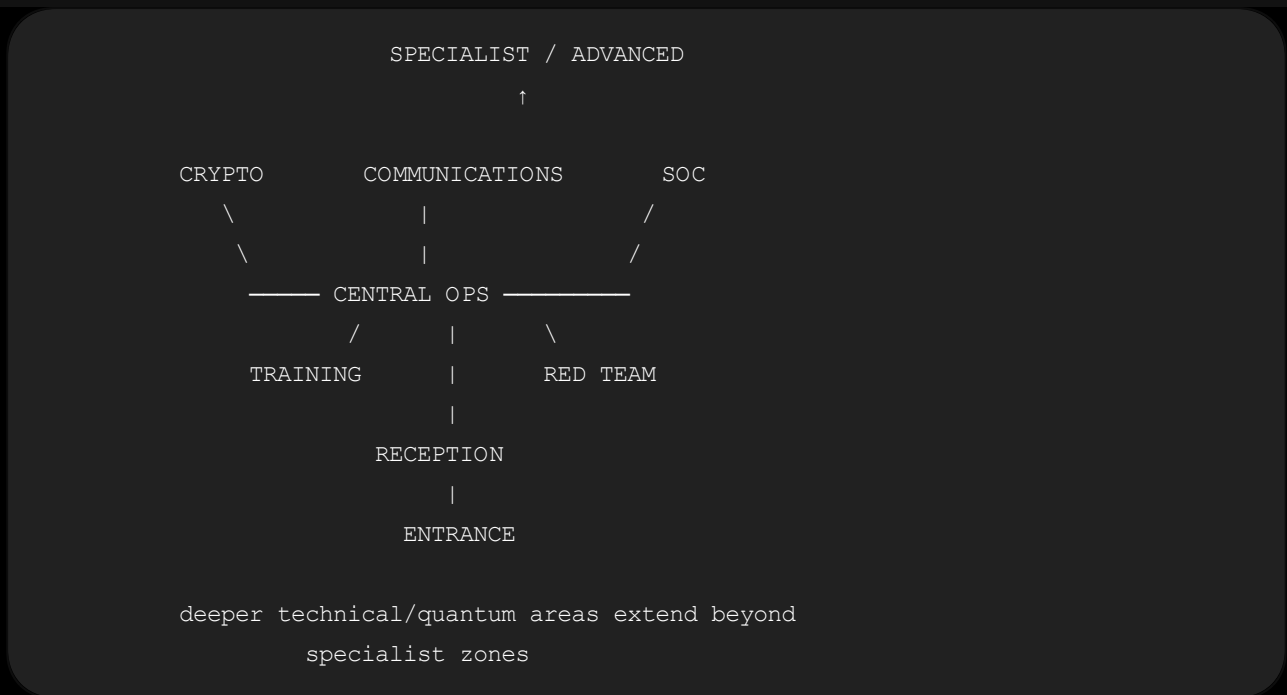
That gives the building meaning.

4. But Central Operations remains the heart

Security depth does not mean Central Operations should sit at the back.

Central Operations should remain the **operational centre** around which the important wings connect.

Think of it more like:



So the player repeatedly returns to a recognisable centre.

That is very useful for navigation.

5. We should design the whole HQ using seven overlays

This is the part I think will stop us going backwards repeatedly.

We draw **one floor plan**, but analyse it seven different ways before accepting it.

Overlay A — Physical architecture

Only:

- rooms;
- walls;
- corridors;
- doors;
- entrances;
- room dimensions;
- structural circulation.

No missions yet.

Question:

Does this work as a believable technical facility?

Overlay B — Campaign flow

Take that exact floor plan and draw:

PROLOGUE

Entrance → Reception → Central Ops

INCIDENT 01

Central Ops → Foundations

INCIDENT 02

Central Ops → Crypto → Communications

INCIDENT 03

Crypto ↔ Communications

INCIDENT 04

Central Ops → Communications → SOC

INCIDENT 05

SOC → Crypto → Communications → Red Team

INCIDENT 06

SOC ↔ Red Team ↔ HQ command/recovery systems
→ Central Ops

Then ask:

Are players constantly walking unreasonable distances?

Does escalation physically feel like escalation?

Are we repeatedly crossing the entire HQ for no reason?

If yes, **the architecture changes now**, not after modelling.

6. Overlay C — Character flow

Then use the same map for:

PLAYER

ALICE

BOB

EVE

RED TEAM

HQ STAFF

For example:

Alice

Needs natural routes between:

```
Central Operations  
Crypto  
Communications
```

She should not need to walk through Red Team Operations just to reach the Crypto Lab.

Eve

Needs plausible access to:

```
Identity/Security  
Communications  
SOC  
security-monitoring spaces
```

without positioning her like a hidden villain.

Bob

Primarily remote, so we need communications/workstation presence rather than constantly physically placing Bob in HQ.

Red Team

Needs:

```
SOC  
Red Team Operations  
HQ authorisation path
```

to make operational sense.

If character routes look stupid, fix the building.

7. Overlay D — Clearance

Now colour the same HQ according to progression.

For example:

GREEN

Open from Prologue

BLUE

Visible / unlocks early

AMBER

Restricted specialist

RED

High-security

PURPLE

Advanced / future release

At the beginning:

Reception

OPEN

Central Ops

LIMITED

Foundations

LOCKED/VISIBLE

Crypto

LOCKED/VISIBLE

SOC

RESTRICTED

Red Team

RESTRICTED

Engineering

RESTRICTED

Quantum

HIGH RESTRICTION

Secure Core

UNKNOWN/RESTRICTED

The player should **see future progression physically**.

A locked glass/security door is more powerful than an icon saying:

Level 4 area unavailable.

8. Overlay E — Gameplay/interactions

Now mark every major interactable.

For example:

CENTRAL OPS

- Workstation 04
- operations display
- communications terminal
- incident status board

FOUNDATIONS

- evidence table
- timeline board
- logic workstation

CRYPTO

- encryption bench
- key station
- asymmetric workbench

SOC

- session monitor
- evidence board
- timeline/correlation station

RED TEAM

- target case console
- authorisation console
- operational terminal
- evidence preservation station

Then ask:

Is each room actually different to play in?

If Crypto Lab and SOC are merely different coloured rooms containing the same generic monitor, we have failed.

9. Overlay F — Camera and handling

Then we put the **60° raised-oblique camera** over the exact building.

We test:

- corridor visibility;
- corners;
- wall occlusion;
- door visibility;
- room depth;
- interaction visibility;
- NPC paths.

This may change the architecture.

For example, a beautiful narrow L-shaped corridor might be architecturally realistic but terrible from our camera.

Then we redesign it.

Gameplay wins.

10. Overlay G — Unity technical structure

Only after the other six overlays work do we convert the map into Unity objects.

Then:

```
HQ_ROOT
|
├─ StructuralShell
├─ ReceptionZone
├─ CentralOpsZone
├─ FoundationsZone
├─ CryptoZone
├─ CommunicationsZone
├─ SOCZone
├─ RedTeamZone
├─ EngineeringZone
├─ QuantumZone
├─ AdvancedComputeZone
└─ SecureCoreZone
```

Those are **technical divisions of one building**.

Not eleven independently designed buildings.

That is the distinction we needed.

11. How we actually build it in Unity

This is where your section approach becomes useful.

First: entire HQ greybox

We should build the **whole Headquarters shell first**.

Not polish it.

Just:

```
ALL ROOMS
ALL CORRIDORS
ALL IMPORTANT DOORS
ALL MAJOR CONNECTIONS
```

using ProBuilder/basic geometry.

Something like:



- Reception
- Central Ops
- Foundations
- Crypto
- Communications
- SOC
- Red Team
- Engineering
- Quantum
- Advanced Compute
- Secure Core

Every department exists from the beginning.

Even if it is literally grey walls.

12. Then perform a whole-building test

Before polishing even Central Operations:

Test 1 — Walk the whole HQ

From entrance to every room.

Test 2 — Run the Prologue route

Entrance → Reception → Central Ops → WS04

Test 3 — Run Incident 01–06 routes

Not the actual gameplay.

Just movement.

Test 4 — Run NPC routes

Alice/Eve/Red Team representative paths.

Test 5 — Test camera

Every corridor and room.

Test 6 — Test clearance

Can future spaces be seen without being accessible?

Test 7 — Test distance

Is anything annoyingly far?

Only after those tests pass:

HQ MASTER GREYBOX LOCK

13. Then we section the construction

Now we use the controlled sections you proposed.

But we are **not designing the sections anymore**.

We are finishing pieces of an already designed whole.

For example:

```
WHOLE HQ GREYBOX
```

```
↓
```

```
CENTRAL OPS
```

```
finish function + interactions
```

```
FOUNDATIONS
```

```
finish function + interactions
```

```
CRYPTO
```

```
finish function + interactions
```

```
SOC
```

```
finish function + interactions
```

Each time we finish a room, it replaces its greybox version **in the same headquarters**.

The overall footprint and connections do not casually change.

14. Same with characters

Characters should not be developed in isolation and later “added to the map.”

We first define their movement requirements on the master HQ.

Then separately build:

```
CharacterBase  
Player  
Alice  
Bob  
Eve  
Red Team
```

Once Alice works, we test her in the **actual whole HQ greybox**.

Example:

```
Alice Spawn  
↓  
Reception corridor  
↓  
Central Ops  
↓  
Workstation 04
```

If her NavMesh movement works there, good.

Later:

```
Alice  
Central Ops → Crypto
```

Same world.

15. Same with room pop-ups

The pop-ups are a separate **system**, but designed against the whole map.

For example:

```
PLAYER CROSSES ROOM BOUNDARY  
  
↓  
  
RoomZone detects:  
HQ_CRYPT0  
  
↓  
  
PHANTOM Q ENGINEERS
```

CRYPTOGRAPHY LAB
Protection & Key Management

↓
fade

Every room uses the same system.

Again:

system developed separately, deployed globally.

16. This should be our philosophy

There are two different meanings of “separate”.

Bad separation

```
Design Central Ops
finish Central Ops
↓
Design Crypto
finish Crypto
↓
Design SOC
↓
eventually try connecting them
```

That is what we should avoid.

Correct separation

```
DESIGN WHOLE HQ
↓
LOCK RELATIONSHIPS
↓
BUILD WHOLE GREYBOX
↓
TEST WHOLE SYSTEM
↓

then separately engineer:

Building system
Room system
Character system
UI system
```

Clearance system
Mission system
etc.

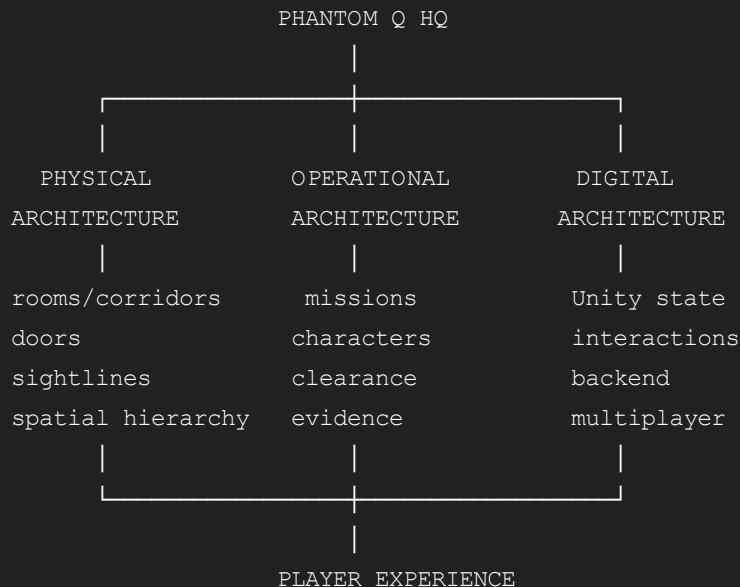
↓

all plug into the SAME HQ

That's the correct architecture.

17. The Headquarters itself becomes a system of systems

This is the mindset I think fits Phantom Q best:



A meaningful Headquarters is produced only when all three agree.

18. Room placement must have reasons

When we eventually draw the actual map, every adjacency needs justification.

For example:

Reception near Entrance

Obvious:

OUTSIDE
↓
IDENTITY



ACCESS CONTROL

Central Operations immediately beyond controlled entry

Because it is the operating centre.

Communications near Central Operations

Because normal operational communication is frequent.

SOC near Communications

Because communication/session evidence escalates into security monitoring.

Red Team near SOC but separately secured

Because:

```
SOC
detect/investigate
↓
RED TEAM
authorised operational response
```

They are related but not the same function.

Crypto near Communications

Because:

```
CRYPTO
protect information

COMMUNICATIONS
move information
```

Their relationship is educationally meaningful.

Quantum/Advanced Compute deeper inside

Because they represent more advanced/high-value capabilities.

Secure Core deepest

Because the architecture should physically imply:

highest trust / greatest restriction.

That's what I mean by making the building meaningful.

19. The campaign should make a spatial loop

Another thing I would intentionally design:

The player begins at:

Central Operations

and after Incident 06 effectively returns there.

So spatially:

```
PROLOGUE
CENTRAL OPERATIONS
normal operations
    ↓

player expands outward through HQ

Foundations
Crypto
Communications
SOC
Red Team
    ↓

INCIDENT 06
recovery / restoration
    ↓

CENTRAL OPERATIONS
normal state restored
```

That's a very satisfying environmental story.

The room from the beginning now has different meaning at the end.

20. Environmental state should also evolve

Same building.

Different state.

For example Central Ops:

Prologue

NORMAL
white/blue operations displays
routine staff movement

PQ-001

LOCAL INCIDENT
status indicator

Incident 04

RESTRICTED
SOC escalation
financial impact
affected channel

Incident 06

RECOVERY ACTIVE
mission status

Ending

RESTORED
incident closed
actor unresolved

We do not create five different Central Operations rooms.

We create **one persistent room with state**.

That will make Phantom Q feel like an actual world.

21. So I would restructure our planning into four master phases

Not ten disconnected sections.

PHASE A — DESIGN THE WHOLE SYSTEM

We solve together:

```
A1 Building meaning
A2 Spatial hierarchy
A3 Department adjacency
A4 Whole HQ floor plan
A5 Campaign routes
A6 Character routes
A7 Clearance progression
A8 interaction distribution
A9 camera/handling
A10 Unity partition
```

Output: one canonical HQ specification.

PHASE B — BUILD THE WHOLE GREYBOX

Unity:

```
complete shell
all rooms
all corridors
all doors
placeholder objects
NavMesh
player
camera
room zones
```

Output: fully walkable Phantom Q HQ.

Nothing is pretty yet.

PHASE C — BUILD SYSTEMS IN CONTROLLED SECTIONS

Now separately solve:

```
Characters
Room UI
Interactions
Clearance
Mission engine
Sequences
Workstations
Evidence
Crypto systems
```

But each is tested **inside the whole HQ**.

PHASE D — BUILD THE CAMPAIGN INTO IT

Then:

```
Prologue
↓
Incident 01
↓
02
↓
03
↓
04
↓
05
↓
06
```

Each incident uses the systems already established.

22. This also changes what we should do next

I don't think the next task should be more C# yet.

And I don't think it should be polishing Central Operations.

The next task should be:

PHANTOM Q HQ — MASTER SPATIAL ARCHITECTURE

And we should properly solve, in one pass:

1. Exact required departments
2. Which departments **MUST** connect directly
3. Which **MUST** be separated
4. Security depth
5. Entrance/exit structure
6. Central hub structure
7. Main/secondary corridors
8. Whole-building dimensions
9. Approximate room dimensions

10. Incident routes 0-06
11. Character routes
12. Clearance gates
13. Sightlines/future-area visibility
14. Camera compatibility
15. Future expansion points

Then we draw the actual canonical floor plan.

Once that floor plan passes all those overlays, we build the **entire greybox in Unity**.

That prevents the back-and-forth you're concerned about: the **whole is designed once**, while the sections become controlled engineering packages used to manufacture that whole.